BAPS Unified Backend Architecture Plan

System Blueprint for 'Sampark' (Contact Dictionary) & 'Upasthiti' (Attendance Tracking)

This document presents a comprehensive production-grade architectural specification for a centralized backend system. It serves two applications for the BAPS community: **Sampark** (a robust contact management dictionary) and **Upasthiti** (a dynamic attendance tracking system). This plan utilizes a unified PostgreSQL relational database layer and a high-performance, asynchronous FastAPI backend engine.

1. Executive Summary & Structural Domains

The operational framework of this application mirrors the organization structure of the BAPS volunteer system. To ensure clarity, the programmatic definitions align with the geographic and human workflows described below:

- **Kshetra (ક્ષેત્ર):** The top-level administrative regional division (e.g., Ahmedabad, Mumbai, London) governing all underlying entities.
- **Mandal:** Local localized sectors within a Kshetra where spiritual weekly assemblies (Sabhas) occur.
- **Area:** Concrete geographical zones mapping to both Mandals and Haribhakts to keep track of physical residency and operational distribution.
- **Haribhakt:** The core participant profile containing vital demographic records, custom dynamic tags, skillsets, and availability attributes.
- **Karyakar:** A subset of Haribhakts possessing volunteer duties categorized across a explicit hierarchy of roles.

Hierarchical Role Order (Highest to Lowest Privilege):
`sayojak` → `nirdeshak` → `sah_nirdeshak` → `nirikshak` → `sanchalak` → `sah_sanchalak`

2. Complete PostgreSQL Relational Database Schema

The following full DDL represents the complete structure optimized with necessary relations, foreign key constraints, indexes, and automated audit triggers.

```
-- Core Enumerations
CREATE TYPE volunteer_role AS ENUM (
    'sah_sanchalak', 'sanchalak', 'nirikshak', 'sah_nirdeshak', 'nirdeshak', 'sayojak'
);
CREATE TYPE profile_status AS ENUM (
    'active', 'inactive', 'deceased', 'moved'
);
CREATE TYPE attendance_status AS ENUM (
    'present', 'absent'
);

-- 1. Kshetra Table
CREATE TABLE kshetras (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
```

```
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 2. Areas Table
CREATE TABLE areas (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    kshetra_id INT REFERENCES kshetras(id) ON DELETE RESTRICT,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(name, kshetra_id)
);

-- 3. Mandals Table
CREATE TABLE mandals (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    area_id INT REFERENCES areas(id) ON DELETE RESTRICT,
    kshetra_id INT REFERENCES kshetras(id) ON DELETE RESTRICT,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(name, area_id)
);

-- 4. Users (Authentication Credentials) Table
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    hashed_password VARCHAR(255) NOT NULL,
    role volunteer_role NOT NULL,
    home_mandal_id INT REFERENCES mandals(id) ON DELETE RESTRICT,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 5. Dynamic Tags Table
CREATE TABLE tags (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) NOT NULL UNIQUE,
    is_system BOOLEAN DEFAULT FALSE, -- TRUE = Admin locked, FALSE = Custom Karyakar added
    created_by INT REFERENCES users(id) ON DELETE SET NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 6. Haribhakts Core Table
CREATE TABLE haribhakts (
    id SERIAL PRIMARY KEY,
    photo_url TEXT,
    first_name VARCHAR(100) NOT NULL,
    middle_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    mobile_no VARCHAR(15) NOT NULL UNIQUE,
```

```
    alternate_mobile_no VARCHAR(15),
    email VARCHAR(150),
    dob DATE NOT NULL,
    gender VARCHAR(10) NOT NULL,
    marital_status VARCHAR(20),
    blood_group VARCHAR(5),
    native_village VARCHAR(100),
    address TEXT NOT NULL,
    area_id INT REFERENCES areas(id) ON DELETE RESTRICT,
    mandal_id INT REFERENCES mandals(id) ON DELETE RESTRICT,
    work_business_details TEXT,
    skills TEXT[], -- Array of strings for flexible text-based tagging
    regularity_sabha VARCHAR(50),
    regularity_darshan VARCHAR(50),
    regularity_seva VARCHAR(50),
    notes TEXT,
    status profile_status DEFAULT 'active',
    created_by INT REFERENCES users(id) ON DELETE SET NULL,
    updated_by INT REFERENCES users(id) ON DELETE SET NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 7. Haribhakt Tag Join Table
CREATE TABLE haribhakt_tags (
    haribhakt_id INT REFERENCES haribhakts(id) ON DELETE CASCADE,
    tag_id INT REFERENCES tags(id) ON DELETE CASCADE,
    custom_value VARCHAR(100), -- Dynamic properties (e.g., Instrument -> Tabla)
    PRIMARY KEY (haribhakt_id, tag_id)
);

-- 8. Karyakar Roles Assignment Table
CREATE TABLE karyakars (
    id SERIAL PRIMARY KEY,
    haribhakt_id INT REFERENCES haribhakts(id) ON DELETE CASCADE UNIQUE,
    karyakar_id_code VARCHAR(50) UNIQUE,
    assigned_role volunteer_role NOT NULL,
    mandal_id INT REFERENCES mandals(id) ON DELETE RESTRICT,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 9. Sabha Sessions Table
CREATE TABLE sabha_sessions (
    id SERIAL PRIMARY KEY,
    mandal_id INT REFERENCES mandals(id) ON DELETE RESTRICT,
    session_date DATE NOT NULL,
    time_slot VARCHAR(50) NOT NULL, -- e.g., "Monday 7:00 PM"
    sabha_type VARCHAR(50) DEFAULT 'General',
    created_by INT REFERENCES users(id) ON DELETE SET NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (mandal_id, session_date, time_slot)
);

-- 10. Attendance Records Table
CREATE TABLE attendance (
    id SERIAL PRIMARY KEY,
```

```
    session_id INT REFERENCES sabha_sessions(id) ON DELETE CASCADE,
    haribhakt_id INT REFERENCES haribhakts(id) ON DELETE CASCADE,
    status attendance_status NOT NULL,
    is_visitor BOOLEAN DEFAULT FALSE, -- True if senior karyakar marking across boundary
    marked_by INT REFERENCES users(id) ON DELETE SET NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (session_id, haribhakt_id)
);

-- Performance Enhancing Indexes
CREATE INDEX idx_haribhakts_mandal ON haribhakts(mandal_id);
CREATE INDEX idx_haribhakts_names ON haribhakts(last_name, first_name);
CREATE INDEX idx_attendance_session ON attendance(session_id);
```

3. FastAPI Project Layout Structure

A production modular directory layout scaling cleanly for development teams:

```
baps_backend/
├── app/
│   ├── __init__.py
│   ├── main.py               # App lifecycle initialization and routers aggregation
│   ├── core/
│   │   ├── config.py         # Environment settings, secrets, and constants
│   │   ├── database.py       # SQLAlchemy Async engine configuration & session handling
│   │   └── security.py       # Password hashing, JWT creation & token parsing
│   ├── dependencies/
│   │   └── auth.py           # RBAC hierarchy resolution and current user injections
│   ├── models/               # SQLAlchemy database declarations
│   │   ├── __init__.py
│   │   ├── organization.py   # Kshetra, Mandal, Area tables
│   │   ├── profiles.py       # Haribhakt, Karyakar, Tags tables
│   │   └── operations.py     # SabhaSession, Attendance tables
│   ├── schemas/              # Pydantic parsing and validations
│   │   ├── auth.py
│   │   ├── haribhakt.py
│   │   └── attendance.py
│   └── routers/              # Micro-service route end-points
│       ├── auth.py
│       ├── master.py         # Mandal, Kshetra, and Area CRUD
│       ├── sampark.py        # Complete Haribhakt directory routes
│       └── upasthiti.py      # Attendance tracking mechanisms
├── alembic/                  # Database version migration files
├── .env                      # Environment credentials configuration file
└── requirements.txt          # Package specifications
```

4. Hierarchical Role-Based Access Control (RBAC) Framework

To eliminate manual validation chains across hundreds of endpoints, the backend evaluates role weights as integers. This enforces rigorous security boundaries cleanly:

```
ROLE_WEIGHTS = {
    "sah_sanchalak": 1,
    "sanchalak": 2,
    "nirikshak": 3,
    "sah_nirdeshak": 4,
    "nirdeshak": 5,
    "sayojak": 6
}
```

Data Isolation Boundary Enforcements

Hierarchical Category	Role Context	Permitted Scope Limit
Tier 1 — Local Operators	`sah_sanchalak`, `sanchalak`, `nirikshak`	Can query and write records strictly within their own `home_mandal_id`.
Tier 2 — Regional Directors	`sah_nirdeshak`, `nirdeshak`	Can manage all underlying Mandals within their designated `Kshetra`. Allowed to manage master tags/areas.
Tier 3 — Global Overseers	`sayojak`	Unrestricted data querying and operational modifications across all Kshetras.

5. Implementation Code Snippets

Database Session Infrastructure (`app/core/database.py`)

```
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession, async_sessionmaker
from sqlalchemy.orm import declarative_base
import os

DATABASE_URL = os.getenv("DATABASE_URL", "postgresql+asyncpg://postgres:password@localhost:5432/
baps_db")

engine = create_async_engine(DATABASE_URL, echo=False, future=True)
AsyncSessionLocal = async_sessionmaker(bind=engine, class_=AsyncSession, expire_on_commit=False)
Base = declarative_base()

async def get_db():
    async with AsyncSessionLocal() as session:
        try:
            yield session
            await session.commit()
        except Exception:
            await session.rollback()
            raise
```

RBAC Dependency Guard Engine (`app/dependencies/auth.py`)

```
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from jose import jwt, JWTError
from app.core.security import SECRET_KEY, ALGORITHM
from app.core.database import get_db
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy.future import select
from app.models.organization import User

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="api/auth/login")

ROLE_WEIGHTS = {"sah_sanchalak": 1, "sanchalak": 2, "nirikshak": 3, "sah_nirdeshak": 4,
"nirdeshak": 5, "sayojak": 6}

async def get_current_user(token: str = Depends(oauth2_scheme), db: AsyncSession =
Depends(get_db)):
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Could not validate security tokens",
    )
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        username: str = payload.get("sub")
        if username is None:
            raise credentials_exception
    except JWTError:
        raise credentials_exception

    result = await db.execute(select(User).where(User.username == username))
    user = result.scalars().first()
    if user is None:
        raise credentials_exception
    return user

class RoleRequirementChecker:
    def __init__(self, minimum_allowed_role: str):
        self.minimum_allowed_role = minimum_allowed_role

    def __call__(self, current_user: User = Depends(get_current_user)):
        if ROLE_WEIGHTS[current_user.role.value] < ROLE_WEIGHTS[self.minimum_allowed_role]:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail="Privilege execution limit reached. Access Denied."
            )
        return current_user
```

Core Operations Router (`app/routers/upasthiti.py`)

```
from fastapi import APIRouter, Depends, HTTPException, status
from sqlalchemy.ext.asyncio import AsyncSession
from app.core.database import get_db
from app.dependencies.auth import get_current_user, RoleRequirementChecker
from app.models.organization import User
```

```
from app.models.operations import SabhaSession, Attendance
from pydantic import BaseModel
from typing import List
from datetime import date

router = APIRouter(prefix="/api/upasthiti", tags=["Upasthiti Attendance Core"])

class BulkAttendanceSubmit(BaseModel):
    haribhakt_ids: List[int]
    status: str

@router.post("/sessions/{session_id}/attendance/bulk")
async def bulk_submit_attendance(
    session_id: int,
    payload: BulkAttendanceSubmit,
    db: AsyncSession = Depends(get_db),
    current_user: User = Depends(get_current_user)
):
    # Validate session existence
    session_check = await db.get(SabhaSession, session_id)
    if not session_check:
        raise HTTPException(status_code=404, detail="Sabha session object structural record
missing")

    # Enforce data perimeter boundaries
    if current_user.role.value in ["sah_sanchalak", "sanchalak", "nirikshak"]:
        if session_check.mandal_id != current_user.home_mandal_id:
            raise HTTPException(status_code=403, detail="Cross-mandal operation blocked for this
privilege tier")

    # Efficient localized batch synchronization
    for h_id in payload.haribhakt_ids:
        is_visitor_profile = (session_check.mandal_id != current_user.home_mandal_id)

        # Check or insert logic
        stmt = Attendance(
            session_id=session_id,
            haribhakt_id=h_id,
            status=payload.status,
            is_visitor=is_visitor_profile,
            marked_by=current_user.id
        )
        await db.merge(stmt)

    await db.commit()
    return {"status": "success", "processed_records_count": len(payload.haribhakt_ids)}
```

6. Step-by-Step Production Implementation Guide

Phase 1: Local Setup & Database Provisioning

Initialize your virtual execution runtime and install core dependencies. Create a managed instance of PostgreSQL locally or leverage Supabase for instant visual data validation.

```
python -m venv venv
source venv/bin/activate
pip install fastapi uvicorn sqlalchemy asyncpg alembic python-jose[cryptography] passlib[bcrypt]
pydantic
alembic init alembic
```

Configure the `alembic.ini` database locator string and append your relational models structure within `env.py` to execute your initial automated baseline migration:

```
alembic revision --autogenerate -m "initialize_baps_schema"
alembic upgrade head
```

Phase 2: Core Authentication Layer Establishment

Implement security hashing algorithms within `security.py` using Passlib. Build basic user signup scripts or direct manual database provisions for top-tier administrators. Validate JWT token emission behavior using the interactive API panel (`/docs`).

Phase 3: Sampark Foundation Data Ingestion

Develop standard CRUD functionality for basic entities (Areal zones, Mandals, and dynamic tracking Tags). Verify authorization gates so that only users with `sah_nirdeshak` clear or above can update or delete master values, while local operators keep clear read scopes.

Phase 4: Haribhakt Directory Deployment

Code structural endpoint logic allowing search execution filters on `/api/sampark/haribhakts` by name, contact fields, or specific Mandal allocations. This powers live auto-suggest listings inside the final client application interfaces natively.

Phase 5: Upasthiti Session Synchronization

Deploy endpoints managing active Sabha creations and unified batch submissions. Build localized filters targeting past operational analytics summaries cleanly across any distinct Kshetra.